

数字逻辑期末复习宝典

第一部分：基础理论篇 (第1-3章)

资料来源：《数字逻辑》第七版 (白中英) 教材

第一章 开关理论基础

本章是数字逻辑的基石，重点在于数制转换与逻辑函数的化简。

1. 数制与编码

- 进位计数制：
 - 二进制 (Binary)、八进制 (Octal)、十进制 (Decimal)、十六进制 (Hexadecimal)。
 - 相互转换：
 - 二-十转换：按权展开求和 / 整数除2取余、小数乘2取整。
 - 二-八/十六转换：3位/4位一组，直接映射。
 - 带符号数的表示：
 - 原码：最高位符号位 (0正1负)，其余数值位。
 - 反码：正数同原码；负数符号位不变，数值位按位取反。
 - 补码 (重点)：正数同原码；负数在反码基础上末位+1。
 - 计算公式： $[X]_{\text{补}} = 2^n + X \pmod{2^n}$
 - 补码运算： $[X + Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}}$ (符号位参与运算)。
 - 编码：
 - BCD码 (8421码)：用4位二进制数表示1位十进制数 (0000-1001)，注意修正 (>9加6修正)。
 - 格雷码 (Gray Code)：相邻两数仅1位不同，用于减少过渡误差。
 - ASCII码：字符编码。

2. 逻辑代数基础

- 基本运算：与 (AND)、或 (OR)、非 (NOT)、与非 (NAND)、或非 (NOR)、异或 (XOR)、同或 (XNOR)。
- 基本公式与定理：
 - 摩根定律 (De Morgan's Laws)：
 - $\overline{A \cdot B} = \overline{A} + \overline{B}$
 - $\overline{A + B} = \overline{A} \cdot \overline{B}$
 - 代入定理、反演定理、对偶定理。
- 逻辑函数化简：

- **公式法**：利用并项、吸收、消项、配项等法。
- **卡诺图法 (K-Map)** (核心考点)：
 - **画圈原则**：圈内1的个数为 2^n ；圈越大越好；包含新的1；可以卷绕。
 - **无关项** (Don't Care, $\times$)：可当0或1使用，辅助化简。

第二章 组合逻辑

组合逻辑电路的特点是：**输出仅取决于当前的输入，与电路原来的状态无关（无记忆）。**

1. 组合逻辑分析与设计

- **分析步骤**：电路图 $\rightarrow$ 逻辑表达式 $\rightarrow$ 化简 $\rightarrow$ 真值表 $\rightarrow$ 功能说明。
- **设计步骤**：逻辑命题 $\rightarrow$ 真值表 $\rightarrow$ 逻辑表达式 $\rightarrow$ 化简/变换 $\rightarrow$ 逻辑图。

2. 常用中规模集成电路 (MSI)

- **编码器 (Encoder)**：
 - **普通编码器**。
 - **优先编码器 (74148)**：允许同时输入多个信号，只对优先级最高的编码。
- **译码器 (Decoder)**：
 - **3-8线译码器 (74138)** (重点)：
 - 3个输入 A, B, C ，8个输出 $\overline{Y}_0 \sim \overline{Y}_7$ (低电平有效)。
 - **使能端**： $G_1 = 1, \overline{G_{2A}} = 0, \overline{G_{2B}} = 0$ 时工作。
 - **应用**：实现任意逻辑函数（输出是最小项的非，配合与非门实现逻辑和）。
- **数据选择器 (MUX)**：
 - **4选1 (74153), 8选1 (74151)**。
 - **应用**：实现逻辑函数（将变量接选择端，其余接数据端）。
- **加法器 (Adder)**：
 - **半加器 (HA)**：不考虑低位进位。 $S = A \oplus B, C = AB$ 。
 - **全加器 (FA)**：考虑低位进位。 $S = A \oplus B \oplus C_{in}, C_{out} = AB + (A \oplus B)C_{in}$ 。
 - **多位加法器**：串行进位 vs 超前进位 (Look-ahead Carry)。
- **数值比较器**。

3. 竞争与冒险

- **产生原因**：信号经不同路径传输延迟不同。
- **判别**：逻辑式化简后出现 $A + \overline{A}$ (0型冒险) 或 $A \cdot \overline{A}$ (1型冒险)。
- **消除**：增加冗余项、加滤波电容、使用选通脉冲。

第三章 时序逻辑

时序逻辑电路的特点是：**输出不仅取决于当前输入，还与电路原来的状态有关（有记忆）。**

1. 触发器 (Flip-Flops)

- **基本RS触发器**：有约束条件 $RS = 0$ (或非门构成的) 或 $RS = 1$ (与非门构成的)。
- **钟控触发器**：
 - **D触发器**： $Q^{n+1} = D$ (特性方程)。
 - **JK触发器**： $Q^{n+1} = J\overline{Q}^n + \overline{K}Q^n$ (万能触发器)。
 - **T触发器**： $Q^{n+1} = T \oplus Q^n$ 。
- **触发方式**：电平触发 (可能有空翻)、边沿触发 (上升沿/下降沿)。

2. 时序电路分析

- **步骤**：电路图 $\rightarrow$ 驱动方程 $\rightarrow$ 状态方程 $\rightarrow$ 输出方程 $\rightarrow$ 状态转换表/图 $\rightarrow$ 时序图 $\rightarrow$ 功能分析。
- **模型**：
 - **Moore型**：输出只与状态有关 ($Y = F(Q)$)。
 - **Mealy型**：输出与状态和输入都有关 ($Y = F(Q, X)$)。

3. 时序电路设计

- **步骤**：逻辑抽象 $\rightarrow$ 状态化简 $\rightarrow$ 状态分配 $\rightarrow$ 选触发器求方程 $\rightarrow$ 画逻辑图 $\rightarrow$ 检查自启动。

4. 常用时序逻辑部件

- **寄存器**：
 - **数据寄存器**：并行存取。
 - **移位寄存器**：左移、右移、串入并出、并入串出。
- **计数器**：
 - **同步 vs 异步**：时钟脉冲是否同时作用于所有触发器。
 - **模N计数器**：利用复位法或置位法将模M计数器 (如74161) 改为模N。
 - **集成计数器 (74161/74160)**：具有预置数、清零、使能功能。

第二部分：系统与逻辑篇 (第4-6章)

资料来源：课件 PDF (数字电路4(2), 5(1), 6-1(1), 6-2(1))

第四章 存储系统

- **存储系统的层次结构**：
 - **Cache** (速度最快, 容量小) $\rightarrow$ **主存储器** (内存) $\rightarrow$ **辅助存储器** (磁盘、光盘, 容量大, 非易失)。
 - 关注指标：速度、容量、价格、访问方式。
- **半导体存储器分类**：
 - **RAM (随机存取存储器)**：掉电丢失。

- **SRAM (静态)**: 速度快, 不用刷新, 常作Cache。
- **DRAM (动态)**: 集成度高, 需刷新, 常作主存。
- **ROM (只读存储器)**: 非易失。
 - **Mask ROM** (掩膜)、**PROM** (一次编程)、**EPROM** (紫外线擦除)、**EEPROM** (电擦除)、**Flash Memory** (闪存)。
- **存储容量扩展**:
 - **位扩展**: 增加数据位宽 (e.g., $1K \times 4 \rightarrow 1K \times 8$, 需2片并联)。
 - **字扩展**: 增加存储单元数量 (e.g., $1K \times 8 \rightarrow 2K \times 8$, 需2片, 利用高位地址做片选)。
 - **字位同时扩展**。
 - **课件思考题**:
 - $32K \times 16$ 意味: 15根地址线 ($2^{15} = 32K$), 16根数据线。
 - 构成 $4M \times 32$ 存储器需要 $16K \times 8$ 的芯片多少片?
 - 容量计算:

$$(4M \times 32) / (16K \times 8) = (4 \times 1024 / 16) \times (32 / 8) = 256 \times 4 = 1024$$
 片。
- **地址译码**: 使用译码器 (如2:4或3:8) 控制存储芯片的片选端 (CS/CE)。

第五章 可编程逻辑器件 (PLD)

- **PLD发展历程**:
 - PROM/PLA/PAL/GAL (低密度, SPLD) $\rightarrow$ CPLD/FPGA (高密度)。
 - **PLA**: 与阵列、或阵列均可编程。
 - **PAL**: 与阵列可编程、或阵列固定。
 - **GAL**: 通用阵列逻辑, 输出逻辑宏单元 (OLMC) 可配置, 电可擦除。
- **FPGA (现场可编程门阵列)**:
 - **基本结构**:
 1. **CLB (可编程逻辑块)**: 核心是 **LUT (查找表)**, 实质是 16×1 的SRAM, 配合触发器构成。
 2. **IOB (可编程I/O模块)**: 处理输入输出。
 3. **Interconnect (互连资源)**: 可编程布线。
 - **特点**: 基于SRAM (掉电丢失, 需配置芯片), 集成度极高, 开发周期短。
- **CPLD (复杂可编程逻辑器件)**:
 - 基于乘积项 (Product Term) 结构, 类似大容量PAL/GAL。
 - 非易失性 (EEPROM/Flash工艺)。

第六章 数字系统

1. 数字系统的基本概念

- **五大部件**：输入设备、输出设备、**存储器**、**运算器**、**控制器**。
- **结构**：**数据通路 (Data Path)** + **控制逻辑 (Control Logic)**。
 - **数据通路**：执行部件（ALU、寄存器、总线），处理信息。
 - **控制器**：发出命令，指挥协调。

2. 控制器设计 (ASM图法)

- **ASM图 (算法状态机图)**：
 - **状态框** (矩形)：表示系统状态，包含该状态下的输出（Moore型输出）。
 - **判断框** (菱形)：根据输入条件判断跳转。
 - **条件输出框** (平行四边形)：在特定状态且满足特定条件时的输出（Mealy型输出）。
- **设计步骤**：
 1. **逻辑划分**：确定数据通路和控制器。
 2. **画ASM图**：描述控制流程。
 3. **状态编码**：给每个状态分配二进制码。
 4. **求状态转换和输出逻辑**：
 - 列状态转移表或直接由ASM图写出逻辑表达式。
 - **课件实例**: $C_1 = Q_1^n Q_0^n$ (状态译码)。
 5. **电路实现**：使用触发器（状态寄存）+ 组合逻辑（下一态逻辑 & 输出逻辑）。
- **控制器的分类**：
 - **计数型**：适合状态顺序跳转。
 - **定序型 (One-hot)**：用N个触发器表示N个状态（如状态0: 0001, 状态1: 0010），译码简单但由触发器多。
 - **MUX型**：利用多路选择器实现次态逻辑。

第三部分：Verilog HDL篇

资料来源： 课件 PDF (Verilog-1, Verilog-2)

1. 基础语法与结构

- **模块 (Module)**：Verilog的基本设计单元。

```
module module_name (port_list);  
    // 声明与逻辑  
endmodule
```

- **数据类型**：

- **wire**: 线网型, 用于连接, 常配合 `assign` 使用。
- **reg**: 寄存器型, 用于 `always` 块中保持数据 (不一定对应物理寄存器)。
- **逻辑值**: 0 (低电平), 1 (高电平), x (不定), z (高阻态)。
- **常量表示**: 位宽'进制数值, 如 `8'b10100011` (8位二进制), `8'hA3` (8位十六进制)。

2. 建模方式

- **数据流建模**: 使用 `assign` 语句。
 - `assign y = a & b;` (描述组合逻辑, 并发执行)。
- **行为建模**: 使用 `always` 块。
 - `always @(敏感信号列表)`
 - **组合逻辑**: `always @(*)` 或 `always @(所有输入)`, 使用阻塞赋值 `=`。
 - **时序逻辑**: `always @(posedge clk or negedge rst_n)`, 使用非阻塞赋值 `<=`。
- **结构化建模**: 模块实例化 (Instantiation)。

3. 核心语句

- **if-else**: 必须在 `always` 块内使用。
- **case**: 多分支选择, 必须在 `always` 块内。
 - **重要原则**: 组合逻辑设计时, 若 `if` 不全或 `case` 无 `default`, 会生成**锁存器 (Latch)** (通常要避免)。
- **阻塞 vs 非阻塞赋值**:
 - **阻塞赋值 (=)**: 顺序执行, 用于组合逻辑。
 - **非阻塞赋值 (<=)**: 并行执行, 用于时序逻辑 (描述D触发器行为)。

4. 经典电路代码 (背诵/理解)

- **D触发器 (带异步复位)**:

```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        q <= 1'b0;
    else
        q <= d;
end
```

- **计数器**:

```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) count <= 0;
    else if (en) count <= count + 1;
end
```

- **4选1多路选择器 (MUX4):**

```
always @(*) begin
    case (sel)
        2'b00: out = in0;
        2'b01: out = in1;
        2'b10: out = in2;
        default: out = in3;
    endcase
end
```

- **三态门:** `assign out = en ? data : 1'bz;`

5. 状态机 (FSM) 设计

- **三段式写法 (推荐):**

1. **CS (Current State):** 时序逻辑，更新当前状态。
2. **NS (Next State):** 组合逻辑，根据输入和CS计算NS。
3. **Output:** 组合逻辑或时序逻辑，产生输出。

- **课件示例:** 累加器控制器的设计，需要定义状态参数 (`parameter S0=..., S1=...`)。